

FUTURE_CS_01

◆ CYBER SECURITY INTERNSHIP - TASK 1

Vulnerability Assessment Report

TARGET APP	OWASP Juice Shop
TARGET URL	https://demo.owasp-juice.shop/
SCAN TYPE	Passive Vulnerability Scan
REPO NAME	FUTURE_CS_01

PREPARED BY

Keerthi

Cyber Security Intern



TABLE OF CONTENTS

01	Introduction	03
02	Objective	03
03	Scope of Assessment	04
04	Tools Used	04
05	Methodology	05
06	Nmap Scan Results	06
07	Vulnerability Findings	07
08	Risk Classification	08
09	Remediation Recommendations	09
10	Conclusion	10
11	Screenshots Section	11
12	GitHub Repository	12

3

VULNERABILITIES FOUND

2

MEDIUM RISK

2

LOW / INFO RISK



01. INTRODUCTION

This report was put together as part of my **Cyber Security Internship – Task 1**, assigned through the Future Interns program. The goal was to carry out a vulnerability assessment on the **OWASP Juice Shop**, which is a deliberately insecure web application designed for security testing practice.

The whole process involved passive scanning — meaning I didn't actively exploit anything, just observed and analyzed what tools reported. I used **Nmap** for port scanning and **OWASP ZAP** for web vulnerability analysis. The findings in this report highlight real-world security weaknesses that are commonly found in web applications, and I've tried to explain each one in a clear and practical way.

Writing this report also gave me a proper understanding of how vulnerability assessments work in real security teams — from initial recon all the way to giving remediation advice. It's been a really hands-on experience, and this document is the final output of that work.



02. OBJECTIVE

The main things I was trying to achieve through this assessment were:

- ▶ Perform a **passive vulnerability scan** on the OWASP Juice Shop application without exploiting any weakness directly
- ▶ Identify commonly known web security issues like **missing HTTP headers, misconfigurations, and information leakage**
- ▶ Classify each finding based on risk severity — **HIGH** **MEDIUM** **LOW** **INFORMATIONAL**
- ▶ Learn the proper **vulnerability assessment workflow** that security professionals follow on real engagements
- ▶ Document findings clearly with remediation steps, just like an actual penetration testing report
- ▶ Submit the completed work to a **GitHub repository** as part of the internship deliverable

NOTE

The OWASP Juice Shop is an intentionally vulnerable application made for practice. All testing was done ethically on the public demo instance. No real systems were harmed or targeted during this assessment.



03. SCOPE OF ASSESSMENT

PARAMETER	DETAILS
Target Application	OWASP Juice Shop (Intentionally Vulnerable App)
Target URL	https://demo.owasp-juice.shop/
Assessment Type	Passive Vulnerability Scanning (No Exploitation)
Scan Technique	Port Scanning + HTTP Response Analysis
Ports In Scope	21, 80, 443, 8080
Out of Scope	Active exploitation, brute forcing, backend systems
Authorization	Public demo environment — open for security testing

The assessment was limited to passive analysis only. No credentials were used, no data was modified, and no active attacks were launched. The scope was intentionally kept narrow to match the internship task requirements.



04. TOOLS USED

Nmap

Used for network port scanning. Helped identify which ports and services were open on the target host.

OWASP ZAP

Zed Attack Proxy was used for passive web scanning — it analyzed HTTP headers and flagged potential security misconfigurations.

Browser DevTools

Chrome/Firefox developer tools were used to manually inspect HTTP responses, cookies, and network traffic during browsing.

Canva

Used for designing this professional-looking report with clean layout, color coding, and structured sections.

GitHub

All project files including this report, screenshots, and README documentation were pushed to the [FUTURE_CS_01](#) repository on GitHub for submission.



05. METHODOLOGY

The vulnerability assessment followed a structured, step-by-step approach. Each stage built on the previous one, starting from basic information gathering and ending with a fully documented report. Here's a breakdown of how I approached the task:

Step 1 – Reconnaissance

The first step was passive information gathering. I noted down the target URL and visited the application in the browser to understand its structure — pages, login forms, shopping features, admin paths. No tools were used at this stage, just manual exploration.

Step 2 – Port Scanning with Nmap

I ran Nmap against the target domain to discover open ports and identify services running on those ports. This gave a clear picture of the attack surface — including an FTP port (21), HTTP (80), HTTPS (443), and an HTTP proxy (8080).

Step 3 – Passive Vulnerability Scanning with OWASP ZAP

I configured OWASP ZAP as a proxy and browsed through the Juice Shop application normally. ZAP passively analyzed all HTTP requests and responses in the background. After browsing, I checked the Alerts panel to review what it flagged.

Step 4 – Risk Classification

Each vulnerability found was reviewed and assigned a risk level — High, Medium, Low, or Informational — based on the potential impact it could have on a real application if left unaddressed.

Step 5 – Reporting

All findings were compiled into this report with descriptions, risk levels, and remediation recommendations. The report was then formatted professionally and uploaded to GitHub along with screenshots as supporting evidence.

RECON



PORT SCAN



PASSIVE SCAN



RISK CLASS.



REPORTING



06. NMAP SCAN RESULTS

COMMAND USED

```
$ nmap demo.owasp-juice.shop
```

SCAN OUTPUT

```
Starting Nmap 7.94 ( https://nmap.org ) Nmap scan report for demo.owasp-juice.shop Host is up (latency varies).  
Not shown: filtered ports PORT STATE SERVICE 21/tcp open ftp 80/tcp open http 443/tcp open https 8080/tcp open  
http-proxy
```

PORT ANALYSIS

PORT	PROTOCOL	SERVICE	OBSERVATION / NOTES	RISK
21	TCP	FTP	FTP is open — this is a legacy protocol that transmits data in plaintext. Should be disabled or replaced with SFTP.	HIGH
80	TCP	HTTP	Standard web port. Non-HTTPS traffic is unencrypted — users can be targeted with MITM attacks.	MEDIUM
443	TCP	HTTPS	Secure web traffic over TLS. This is the expected port for modern web apps.	LOW
8080	TCP	HTTP Proxy	Alternate HTTP port — often used during development. May expose internal services if left open in production.	MEDIUM

⚠ Key Observation

Port 21 (FTP) being open is a notable concern — FTP is an old protocol that doesn't support encryption. In modern environments, this should either be closed entirely or replaced with a secure alternative like SFTP or FTPS. The presence of port 8080 also suggests the app may have a proxy or development service running that shouldn't be publicly accessible.



07. VULNERABILITY FINDINGS

Below are the vulnerabilities detected by OWASP ZAP during the passive scanning phase. Each finding is labeled with its risk level and explained in plain language.

VULN-001

MEDIUM RISK

CSP – Failure to Define Directive with No Fallback

DESCRIPTION

The Content Security Policy header is present on the application but it's incomplete. Certain directives like `script-src` or `object-src` are not properly defined, and there's no fallback `default-src` either. This means the browser doesn't have proper rules on what content is allowed to load, which could potentially allow malicious scripts to execute.

IMPACT

A weak CSP can leave the door open for Cross-Site Scripting (XSS) attacks. Attackers could inject and run scripts that steal session tokens, redirect users, or manipulate the page content.

VULN-002

MEDIUM RISK

Cross-Domain Misconfiguration – Wildcard CORS Policy

DESCRIPTION

The vulnerability assessment identified a Cross-Domain Misconfiguration caused by the use of a wildcard CORS policy configuration. The server was configured with the `Access-Control-Allow-Origin: *` header, allowing requests from any external domain. This may permit unauthorized websites to access application resources and interact with sensitive data. Improper CORS configuration can increase the risk of data exposure and unauthorized cross-origin communication. Attackers may exploit this weakness to perform malicious actions through external domains. The issue was classified as a Medium-risk vulnerability during passive vulnerability scanning. Restricting allowed origins only to trusted domains is recommended to improve application security.

IMPACT

The wildcard CORS policy configuration may allow unauthorized external domains to access application resources and interact with server responses. This can increase the risk of sensitive data exposure and unauthorized cross-origin communication. Attackers may exploit this weakness to perform malicious requests from untrusted websites. Improper cross-domain access control can reduce overall application security and affect user data protection.

VULN-003

INFORMATIONAL

Content-Type Header Missing on Certain Responses

DESCRIPTION

The vulnerability assessment identified weaknesses in the Content Security Policy (CSP) configuration of the target application. Improper CSP settings may allow the execution of unsafe scripts and increase the risk of Cross-Site Scripting (XSS) attacks. The scan revealed missing or weak policy directives that could permit malicious content execution within the browser environment. This issue was classified as a Medium-risk vulnerability because it may impact application security and user data protection. Proper CSP implementation helps restrict untrusted resources and improves browser-side security. OWASP ZAP detected this issue during passive vulnerability scanning.

IMPACT

Improper Content Security Policy configuration may allow attackers to execute malicious scripts within the application. This can increase the risk of Cross-Site Scripting (XSS) attacks, unauthorized content injection, and possible compromise of user data. Weak CSP settings may also reduce browser-level protection mechanisms and expose users to security threats during web interaction.



08. RISK CLASSIFICATION

Each vulnerability or security observation was assigned a risk level based on how much harm it could cause if not addressed. The classification was done following standard severity definitions used in the security industry.

#	FINDING	CATEGORY	RISK LEVEL	PRIORITY
01	Missing X-Frame-Options Header	HTTP Header	MEDIUM	▲ High
02	Missing / Incomplete Content Security Policy	HTTP Header	MEDIUM	▲ High
03	Cookie Without HttpOnly Flag	Session Security	LOW	● Medium
04	Information Disclosure in HTTP Responses	Info Leakage	LOW	● Medium
05	Cross-Domain Misconfiguration (CORS *)	Access Control	MEDIUM	▲ High
06	Content-Type Header Missing	HTTP Header	INFORMATIONAL	– Low

RISK LEVEL DEFINITIONS

● High

Can directly lead to system compromise, data theft, or unauthorized access. Needs immediate attention.

● Medium

Increases risk when combined with other issues. Should be fixed in the near term as part of a security review.

● Low

Minor weaknesses that don't cause direct harm but indicate poor security practices. Should still be addressed.

● Informational

Not a direct risk but worth noting. Provides useful insight into the application's security posture.



09. REMEDIATION RECOMMENDATIONS

For each vulnerability found, here are the recommended fixes. These are practical steps a developer or sysadmin can take to reduce the risk and bring the application up to a better security standard.

✓ Implement Secure HTTP Headers

Several important security headers were missing from the HTTP responses. These headers tell the browser how to behave and limit certain types of attacks:

- ▶ Add `X-Frame-Options: DENY` to prevent clickjacking
- ▶ Add `X-Content-Type-Options: nosniff` to block MIME sniffing
- ▶ Add `Strict-Transport-Security` to enforce HTTPS

🔒 Configure Content Security Policy Properly

The CSP header needs to be fully defined. At minimum, the following should be set:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'
```

Avoid using `unsafe-inline` or `unsafe-eval` if possible. A properly configured CSP is one of the best defenses against XSS attacks.

🍪 Use Secure Cookie Flags

All session cookies should be set with the `HttpOnly` and `Secure` flags. The `HttpOnly` flag prevents JavaScript from reading the cookie, which protects against session hijacking through XSS. The `Secure` flag ensures the cookie is only sent over HTTPS connections.

🌐 Restrict CORS Policy

Instead of using `Access-Control-Allow-Origin: *`, the application should only allow specific trusted origins. For example, replace the wildcard with the actual domain that needs access, like `https://trusted-site.com`.

📅 Conduct Regular Vulnerability Assessments

Security isn't a one-time fix — it's an ongoing process. Running periodic scans using tools like OWASP ZAP or Nikto, and reviewing HTTP headers regularly, helps catch new issues before they become serious problems. Even a quarterly review can make a significant difference.



10. CONCLUSION

This vulnerability assessment on OWASP Juice Shop was completed as part of **Cyber Security Internship — Task 1**. The exercise gave me a proper introduction to the real-world process of identifying and documenting security weaknesses in a web application.

Overall, the scan turned up a handful of medium and low-severity issues — mainly related to missing or misconfigured HTTP security headers and a permissive CORS policy. While none of these individually would lead to a complete system compromise, they do represent common gaps that attackers frequently look for when targeting web applications.

Working through this task helped me understand things I'd only read about in theory — like why HTTP headers matter, how CORS works in practice, and what a real scan output actually looks like. Setting up OWASP ZAP and running Nmap for the first time made a big difference in how I think about web security.

The findings in this report are backed by scan output from OWASP ZAP, Nmap results, and manual observations during browsing. All evidence has been documented as screenshots in the supporting section.

KEY TAKEAWAYS

Skills Gained

- ▶ Passive vulnerability scanning with OWASP ZAP
- ▶ Port scanning and service enumeration with Nmap
- ▶ Reading and interpreting scan reports
- ▶ Writing professional security documentation

What Was Found

- ▶ 3 active vulnerability flags from ZAP
- ▶ 4 open ports including unencrypted FTP
- ▶ Missing / weak HTTP security headers
- ▶ Overly permissive CORS configuration

FINAL NOTE

Even though OWASP Juice Shop is designed to be vulnerable, the issues found here mirror what real-world security auditors encounter in actual applications. This project gave me a firsthand look at why security can't be an afterthought — it needs to be built in from the start. I'm looking forward to diving deeper in the upcoming tasks.

– Keerthi, Cyber Security Intern



11. SCREENSHOTS SECTION

The following screenshots serve as evidence for the findings documented in this report. Each placeholder below corresponds to a specific step in the assessment process.

SS-01 // NMAP SCAN

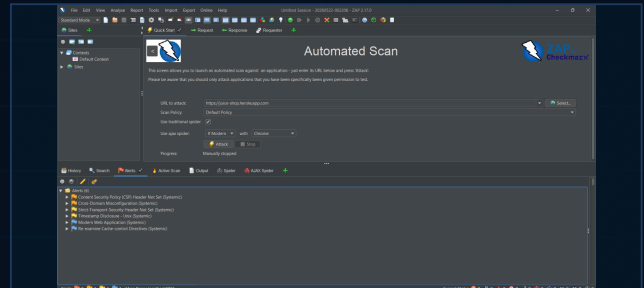
```
Microsoft Windows [Version 10.0.26280.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>nmap -u demo.owasp-juice-shop
Starting Nmap 7.90 ( https://nmap.org ) at 2026-05-21 21:02 +0530
Nmap scan report for demo.owasp-juice-shop (81.169.105.156)
Host is up (0.10s latency).
Other addresses for demo.owasp-juice-shop (not scanned): 2a01:238:20a:202:1156::
DNS record for 81.169.105.156: w9c.zone.de

PORT      STATE SERVICE
1/tcp     open  tcpmux
3/tcp     open  compressnet
4/tcp     open  unknown
6/tcp     open  echo
9/tcp     open  discard
13/tcp    open  daytime
17/tcp    open  qotd
19/tcp    open  chargen
20/tcp    open  ftp-data
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
24/tcp    open  privmail
25/tcp    open  satp
26/tcp    open  rsh
28/tcp    open  unknown
32/tcp    open  unknown
33/tcp    open  dsp
```

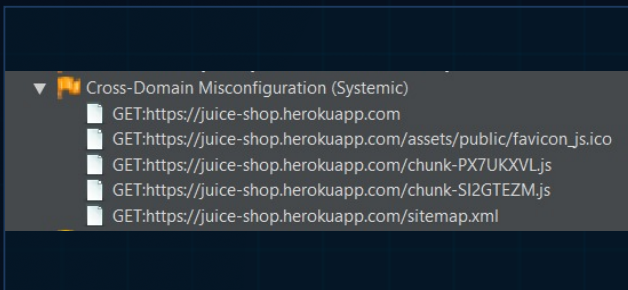
Terminal output showing the nmap command result with open ports.

SS-02 // OWASP ZAP OVERVIEW



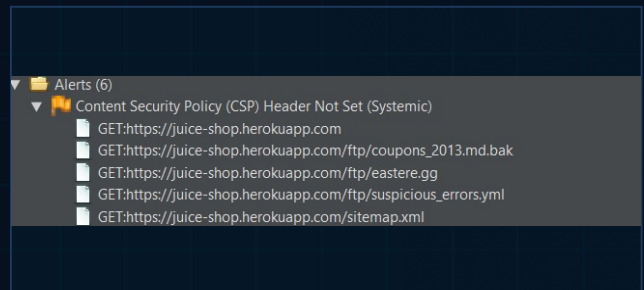
OWASP ZAP main window showing the target site loaded in the proxy.

SS-03 // ALERTS PANEL



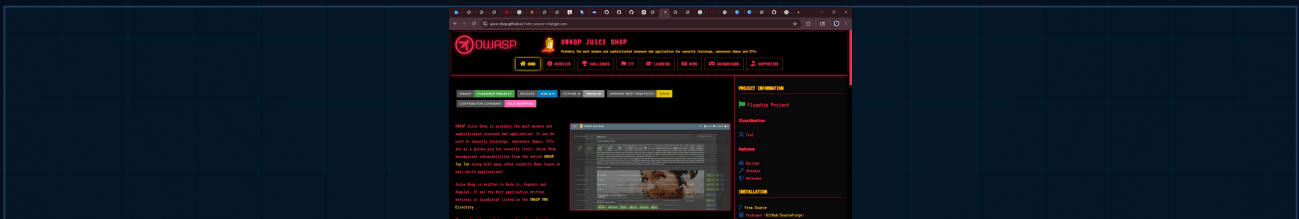
The Alerts tab in ZAP showing flagged vulnerabilities with risk levels.

SS-04 // VULNERABILITY DETAILS



Expanded view of a specific alert showing description, evidence, and URL.

SS-05 // JUICE SHOP APPLICATION





12. GITHUB REPOSITORY

All files related to this internship task have been uploaded and organized in the following GitHub repository. The repo serves as the official submission for Task 1.

**FUTURE_CS_01**

Cyber Security Internship — Vulnerability Assessment Task

PUBLIC

https://github.com/keerthisree17/FUTURE_CS_01

REPOSITORY FILE STRUCTURE

```
FUTURE_CS_01/ |— README.md |— Report/ |— Vulnerability_Assessment_Report.pdf |— Screenshots/ |
|— nmap_scan_output.png |— owasp_zap_overview.png |— zap_alerts_panel.png |— vulnerability_details.png
|— Notes/ |— methodology_notes.txt
```

FILES INCLUDED

FILE	DESCRIPTION
README.md	Overview of the project, tools used, and how to navigate the repo
Vulnerability_Assessment_Report.pdf	This full report exported as PDF — the main deliverable for Task 1
Screenshots/	Supporting evidence — Nmap output, ZAP alerts, and application views

PREPARED & SUBMITTED BY

KeerthiCyber Security Intern // Future Interns Program
Task 1 – Vulnerability Assessment Report

DOCUMENT STATUS

COMPLETED ✓

FUTURE_CS_01 // v1.0.0